MODULE 6

REST principles & resource design

Predictable URLs, HTTP verbs, honest status codes

Routing

```
[Route("api/courses")]
[HttpGet]        // GET collection
[HttpGet("{id}")] // GET one
[HttpPost]       // POST create
// Nouns in path  verbs in HTTP
```

The predictable contract

Session goals

By the end of this session, you will:

- Explain the six REST constraints and why they matter for decoupling
- Design resource-oriented URLs nouns, not verbs
- Use correct success codes (201 Created vs 200 OK, 204 where appropriate)
- Map CRUD to HTTP methods predictably
- Eliminate the 200 OK for errors anti-pattern

Before vs after REST naming

RPC-style (avoid)

- GET /GetAllStudents
- GET /FetchCourseById?id=42
- POST /CreateNewEnrollment
- DELETE /DeleteStudent?id=99
- Verbs in path every URL is a guess

Resource-oriented

- GET /api/students list
- GET /api/courses/42 one
- POST /api/enrollments create
- DELETE /api/students/99 delete
- Noun = resource; verb = HTTP method

Rule of thumb

The URL identifies the resource. The HTTP method identifies the action.

The six REST constraints

Why is statelessness so important for a high-traffic TMS? (Hint: which server in a cluster may handle the next request?)

- Client-server UI does not own the database
- Stateless each request carries what the server needs to act
- Cacheable responses may be reused by intermediaries
- Uniform interface predictable nouns + methods
- Layered system proxies and load balancers stay transparent
- Code-on-demand optional; rarely emphasized for APIs

Status code semantics

Category	Meaning	TMS examples
2xx	Success	200 GET, 201 POST create, 204 PUT/DELETE
3xx	Redirection	Moved resources
4xx	Client error	400 validation, 404 missing, 409 business rule
5xx	Server error	Unhandled failure fix server

Never return 200 for failure

Course is full, but the API returns 200 OK with body text "Course is full". What does Angular HttpClient do?

TypeScript

```typescript
// Angular  200 always hits next()

this.http.post('/api/enrollments', data).subscribe({

  next: () => this.showSuccess(),

  error: () => this.showError()

});
```

C#

```csharp
// Business rule  use 409 Conflict

if (course.IsFull)

  return Conflict(new ProblemDetails {

    Title = "Course at capacity",

    Detail = $"Course '{course.Code}' reached max {course.MaxCapacity}
students.",

    Status = StatusCodes.Status409Conflict

  });
```

Distinction

400 Bad Request invalid input shape. 409 Conflict business rules (full course, duplicate code).

CoursesController resource root

C#

```
[ApiController]

[Route("api/courses")]

public class CoursesController(

  ICourseService svc) : ControllerBase
{

  [HttpGet]

  public Task<IActionResult> GetAll() => ...;


  [HttpGet("{id}")]

  public Task<IActionResult> Get(string id) => ...;


  [HttpPost]

  public Task<IActionResult> Create(

    CreateCourseRequest req) => ...;
}
```

Plural collections

api/courses/42 reads as course 42 from the courses collection. Keep controllers thin delegate to services.

Blueprint path, method, status

Resource path

Noun (e.g. /api/courses/5)

HTTP method

GET POST PUT DELETE

Status code

Truth before body

Teaching cue

GET /api/courses/5 means read one course the verb is GET, not GetCourse in the URL.

Lab brief Session 1

- M6-Labs-Presentation Lab Session 1 (REST foundations)
- M6-Lab-Workbook Exercise 1: CoursesController, 201 + Location
- Exercise 3: 409 Conflict for duplicates and full course; nested enrollments route
- Confirm route and status checkpoints before Session 2

Session summary

Idea	Takeaway
REST	Constraints for predictable, scalable client-server APIs
Nouns vs verbs	Resources in paths; actions in HTTP methods
Status codes	First signal never lie with 200 on failure
409 vs 400	Business conflict vs input validation
Plural routes	Industry default for collections

What this looks like in an interview

What is REST? Separation of concerns, nouns in URLs, verbs in methods, and correct status codes especially 201 for create and honest 4xx for failures.

Next: Session 2 contracts & validation

DTOs, mapping, ProblemDetails (RFC 9457), Data Annotations, and separate create vs update shapes M6-Lab-Workbook Exercise 2.